

```
libx11 mesa multilib-devel wget wireless_tools yasm zip
lib32-mesa lib32-mesa-libgl lib32-ncurses lib32-readline
lib32-zlib
```

B2G can be only compiled with *gcc4.6.4*, and because Arch Linux always has bleeding edge software, you will need to install *gcc46-multilib* from AUR. Remember that you will have to edit the *PKGBUILD* and add *staticlibs* to the *options* array, or *gcc* will be unable to compile B2G and give you a ‘*cannot find -lgcc*’ error when compiling. You will also need to add the following to your *.userconfig* file:

```
export CC=gcc-4.6.4
export CXX=g++-4.6.4
```

By default, Arch Linux uses Python3. You'll have to force it to use the old Python2. You can do that by linking the Python2 executable to Python, but this is discouraged since it is considered error-prone. This will also break Python3 if it is installed on your system. A better way is to use *virtualenv/virtualenvwrapper*:

```
sudo pacman -S python-virtualenvwrapper
source /usr/bin/virtualenvwrapper.sh
mkvirtualenv -p 'which python2' firefoxos
workon firefoxos
```

Android will complain that you need *make 3.81* or *make 3.82* instead of 4.0. You can download *make 2.81* from AUR. This will install the *make-3.81* binary on your path; you need to create a *symlink* named *make* by retaining the same location as mentioned in the *PATH* variable for the build to use the correct version:

```
mkdir -p ~/bin
ln -s 'which make-3.81' ~/bin/make
export PATH=~:/bin:$PATH
```

Android also needs the Java6 SDK, and Arch only has Java7. Unfortunately, the AUR build is broken, but you can still download the Java6 SDK and install it manually. You will then need to put it in your path.

```
cp ~/Downloads/jdk-6u45-linux-x64.bin/opt
su
cd /opt
chmod +x jdk-6u45-linux-x64.bin
./jdk-6u45-linux-x64.bin
exit
ln -s /opt/jdk1.6.0_45/bin/java ~/bin/java
```

Gentoo Linux: You need to install *ccache*, a tool for caching partial builds:

```
emerge -av ccache
```

Because *ccache* is known to frequently cause support issues, Gentoo encourages you to use it explicitly and sparingly.

To enable the required use of *ccache*, in the subsequent step of this guide in which the *.build.shscript* is called, Gentoo users should instead run the command with an explicitly extended path, i.e.:

```
PATH=/usr/lib64/ccache/bin:$PATH ./build.sh
```

Install ADB

The build process needs to pull binary blobs from the Android installation on the phone before building B2G (unless you're building the emulator, of course). For this, you will need ADB, the Android Debug Bridge.



Note: Remember that when you start to use ADB, the phone's lock screen will need to be unlocked to view your phone's contents (at least in later versions of the Firefox OS). You'll probably want to disable the lock screen (we'll get to that later in the build instructions).

Install Heimdall

Heimdall is a utility for flashing the Samsung Galaxy S2. It's used by the B2G flash utility to replace the contents of the phone with Firefox OS, as well as to flash updated versions of B2G and Gaia onto the device. You'll need it if you want to install Firefox OS on a Galaxy S2; it is not needed for any other device. For other devices, we build and use the fastboot utility instead.

There are two ways to install Heimdall:

- You can download the code from Github and build it yourself
- Use the package manager to install Heimdall
 - Run the following command in the terminal: “*sudo apt-get install libusb-1.0-0 libusb-1.0-0-dev*”

Configuring ccache

The B2G build process uses *ccache*. The default cache size for *ccache* is 1GB, but the B2G build easily saturates this; so around 10GB is recommended. You can configure your cache by running the following command inside the terminal:

```
ccache -max-size 10GB
```

Enabling remote debugging

Before you plug your phone back into your USB port, put it in USB developer mode. This allows you to debug and flash the phone. To enable developer mode, enable *Remote Debugging* in *Developer Settings*. Once the option is checked, remote debugging is enabled, and you are ready to go.

At this point, connect your phone to your computer via a